27

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

K8s Güvenliği & RBAC: ServiceAccount, Role ve RoleBinding

Kubernetes Security: RBAC (Roles, Bindings & ServiceAccounts)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Holding binası güvenlik protokolü: 1) Personel Kimlik Kartı (ServiceAccount): Uygulamanızın yakasına taktığı kimlik kartıdır. 2) Yetki Kılavuzu (Role): 'Sadece 2. kattaki çay ocağına girebilir, kasaya dokunamaz' kuralları. 3) Yetki Ataması (RoleBinding): O kimlik kartını o kurallara zimmetleyen resmi tutanaktır. Bu üçlü olmadan hiç kimse API Server'a adım atamaz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Skyscraper Security Protocol: 1) Badge (ServiceAccount): The digital ID card worn by your Spring Boot application. 2) Rulebook (Role): 'Allowed to visit the cafeteria on Floor 2, strictly prohibited from opening the executive vault'. 3) Assignment (RoleBinding): The signed HR form linking the Badge to the Rulebook. Without RBAC, nobody touches the API Server!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kubernetes RBAC (Role-Based Access Control) API Server'a gelen istekleri denetler. Bileşenler: 1) `Subject` (Kullanıcı veya ServiceAccount), 2) `API Groups` (, `apps`, `batch`), 3) `Resources` (`pods`, `services`, `configmaps`), 4) `Verbs` (`get`, `list`, `watch`, `create`, `delete`). `Role` sadece belirli bir namespace içinde geçerlidir; `ClusterRole` ise küme genelinde (örn: Node'ları veya tüm namespace'leri okumak için) geçerlidir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes RBAC authenticates and authorizes API requests. Every rule consists of: API Groups, Resources, and HTTP-like Verbs (`get`, `list`, `watch`, `create`, `delete`). A `Role` binds permissions strictly inside a namespace; a `ClusterRole` spans the entire cluster (e.g. cluster-wide node monitoring). ServiceAccount tokens are mounted into pods at `/var/run/secrets/kubernetes.io/serviceaccount`.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# Spring Cloud Kubernetes için Salt-Okunur İzin:
apiVersion: v1
kind: ServiceAccount
metadata:
  name: spring-cloud-sa
  namespace: coffee-shop
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: configmap-reader
  namespace: coffee-shop
rules:
  - apiGroups: [""]
    resources: ["configmaps", "pods"]
    verbs: ["get", "list", "watch"] # Sadece okuma izni ver!
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bind-spring-reader
  namespace: coffee-shop
subjects:
  - kind: ServiceAccount
    name: spring-cloud-sa
roleRef:
  kind: Role
  name: configmap-reader
  apiGroup: rbac.authorization.k8s.io
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Uygulama pod'larına ASLA gereksiz yere `cluster-admin` yetkisi vermeyin! Spring Boot uygulamanızda bir güvenlik açığı (örn: Log4Shell) oluşursa, saldırgan tüm Kubernetes kümenizi ele geçirir (Least Privilege kuralı).

🇬🇧 WATCH OUT

NEVER grant `cluster-admin` to application pods! If an exploit strikes your Java code, the attacker inherits full administrative root over the entire cluster.

🎯 Özet Çıkarımlar

• En Az Ayrıcalık (Least Privilege) prensibiyle sadece gereken izinler verilmelidir.

• ServiceAccount uygulamaların K8s API'si ile güvenli konuşmasını sağlar.

• ClusterRole küme çapında, Role ise sadece tek bir namespace içinde geçerlidir.

🎯 Key Takeaways

• Always follow the Principle of Least Privilege: grant only essential verbs.

• ServiceAccounts provide secure identities for in-cluster API communication.

• ClusterRoles grant cluster-wide authority; Roles enforce namespace isolation.